

PIZZA SALES ANALYTICS

End-to-End SQL Server & Power BI Analytics Project

Project Documentation

A recruiter-facing technical and business documentation package covering data quality, SQL Server analysis, data modelling, DAX, Power BI dashboard design, advanced SQL patterns, and portfolio publishing.

Author	Hanraj Hari Krishna
Primary tools	Microsoft SQL Server • T-SQL • Power BI • DAX • Power Query
Documentation source	Notion — Pizza Sales project workspace
Repository target	GitHub — Pizza-Sales-Analytics
Documentation status	Framework complete; final dashboard evidence and validated business findings to be added

Portfolio principle: numerical business findings are published only after they are validated against the final dataset and dashboard.

1. Project Overview

Pizza Sales Analytics is an end-to-end business intelligence project designed to demonstrate practical Data Analyst capabilities from raw-data validation through SQL-based analysis and interactive Power BI reporting. The project separates the analytical workflow into data quality, SQL preparation/analysis, modelling, DAX, visualization, and business interpretation.

- Validate source data before analysis and document data-quality risks.
- Use SQL Server as the primary analytical layer for reusable business logic.
- Demonstrate aggregation, CTEs, subqueries, joins, window functions, ranking, date analysis, and reusable database objects.
- Build a Power BI reporting layer with KPI measures, time analysis, product/category analysis, and interactive filtering.
- Present the work as a recruiter-facing GitHub portfolio project with reproducible SQL, documentation, dashboard evidence, and a clear analytical story.

2. Business Objectives

- Measure overall revenue, orders, pizza quantity sold, and average order value.
- Identify high- and low-performing pizza products and categories.
- Compare revenue contribution with sales volume to distinguish high-value and high-volume products.
- Analyze monthly performance and month-over-month revenue movement.
- Explore day-of-week order patterns and advanced product-pair behaviour where supported by the dataset.
- Create reusable SQL views/stored procedures and a Power BI dashboard suitable for business review.

3. Technology Stack

Area	Technology
Database	Microsoft SQL Server
Query language	T-SQL
Transformation	Power Query
Visualization	Microsoft Power BI
Calculations	DAX
Documentation	Notion / Markdown / PDF
Version control	GitHub

4. End-to-End Workflow

Raw Data → Data Quality → SQL Server → Advanced SQL → Power Query → Data Model → DAX → Power BI Dashboard → Business Insights

- Load source data into SQL Server.
- Profile nulls, duplicates, invalid dates, invalid quantities/prices, data types, and calculation consistency.

- Clean and standardize fields required for analysis.
- Create reusable analytical queries, views, and stored procedures.
- Connect Power BI to the prepared analytical layer.
- Build a date-aware model and DAX measures.
- Design a three-page dashboard story: Executive Overview, Product Performance, and Sales Trends / Insights.
- Validate dashboard outputs against SQL results before publishing findings.

5. Dataset & Data Dictionary

The working documentation identifies the following source fields. Exact definitions, data types, and allowed values should be finalized against the source file before the final GitHub release.

Field	Role / expected use
pizza_id	Pizza/order-line identifier
pizza_name	Individual pizza/product name
pizza_type	Source product/type attribute; validate actual values
pizza_category	Category dimension
pizza_size	Size dimension
price	Unit/price field; validate business meaning
quantity	Number of pizzas sold
total_price	Source monetary field; validate whether it is unit price or line total
order_id	Order identifier used for distinct-order analysis
order_date	Order date used for time analysis
city / area	Geographic attributes where present and valid

Data-model caution: the source documentation contains both price and total_price. The final revenue formula must be confirmed from the actual dataset so revenue is not accidentally double-counted.

6. Data Quality & Validation

- Null checks for key analytical fields.
- Duplicate-record and duplicate-order investigation.
- Date completeness and valid date-range checks.
- Numeric validation for quantity and price fields.
- Category and size consistency checks.
- Data-type validation, including conversion of unsuitable text types where required.
- Revenue calculation reconciliation between source fields and derived metrics.

```
SELECT *
FROM Pizza_Sales
WHERE pizza_name IS NULL
      OR quantity IS NULL
      OR order_date IS NULL;
```

ntext compatibility lesson: SQL Server does not support direct comparison operators on ntext values in the same way as normal string types. For the basket-analysis self-join, the project converts relevant fields to NVARCHAR. In a production-style model, unsuitable legacy text types should be

corrected during the cleaning/schema stage.

7. SQL Server Analytical Layer

SQL Server is the primary analytical layer. The project documentation covers both core SQL practice and portfolio-ready analytical patterns.

- Filtering and aggregation: SELECT, WHERE, GROUP BY, HAVING, CASE WHEN.
- Reusable logic: views and stored procedures.
- Data types and conversion: CAST / CONVERT / DECIMAL.
- Date analysis: YEAR, MONTH, DATENAME, DATEPART.
- Advanced querying: CTEs, subqueries, derived tables, self-joins.
- Window functions: ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD.
- Top/bottom analysis and parameterized TOP-N procedures.
- Period-over-period analysis and rolling averages.

Example — reusable monthly sales view

```
CREATE VIEW vw_Monthly_Pizza_Sales AS
SELECT
    YEAR(order_date) AS year_name,
    MONTH(order_date) AS month_number,
    DATENAME(MONTH, order_date) AS month_name,
    COUNT(DISTINCT CAST(order_id AS NVARCHAR(50))) AS total_orders,
    SUM(quantity) AS total_sold,
    SUM(quantity * total_price) AS revenue
FROM Pizza_Sales
GROUP BY
    YEAR(order_date),
    MONTH(order_date),
    DATENAME(MONTH, order_date);
```

Example — Top 3 pizzas within each category

```
WITH PizzaRevenue AS
(
    SELECT pizza_category, pizza_name,
        SUM(quantity * total_price) AS revenue
    FROM Pizza_Sales
    GROUP BY pizza_category, pizza_name
),
RankedPizzas AS
(
    SELECT pizza_category, pizza_name, revenue,
        ROW_NUMBER() OVER
            (PARTITION BY pizza_category ORDER BY revenue DESC) AS rn
    FROM PizzaRevenue
)
SELECT pizza_category, pizza_name, revenue
FROM RankedPizzas
WHERE rn <= 3
ORDER BY pizza_category, revenue DESC;
```

8. Advanced SQL Portfolio Patterns

Pattern	Business question	Key SQL technique
Rolling 7-day revenue	Is revenue trending beyond daily noise?	CTE + windowed AVG
Basket analysis	Which pizzas are ordered together?	Self-join + pair de-duplication
Top 3 by category	Which products lead each category?	CTE + ROW_NUMBER + PARTITION BY
MoM revenue growth	How did revenue change vs. prior month?	CTE + LAG + percentage calculation
Day-of-week analysis	Which weekdays generate more order volume?	DATEPART + DATENAME + aggregation

Pattern	Business question	Key SQL technique
Category comparison	How do categories differ in orders, quantity, and revenue	COUNT DISTINCT + SUM + GROUP BY

Important portfolio rule: advanced analyses are analytical patterns until their outputs are executed and validated against the final dataset. Do not publish sample numbers as project findings.

Example — Month-over-Month growth

```
WITH MonthlyRevenue AS
(
    SELECT YEAR(order_date) AS order_year,
           MONTH(order_date) AS order_month,
           SUM(quantity * total_price) AS revenue
    FROM Pizza_Sales
    GROUP BY YEAR(order_date), MONTH(order_date)
),
RevenueWithPrevious AS
(
    SELECT order_year, order_month, revenue,
           LAG(revenue) OVER
             (ORDER BY order_year, order_month) AS previous_month_revenue
    FROM MonthlyRevenue
)
SELECT order_year, order_month, revenue,
       previous_month_revenue,
       ROUND((revenue - previous_month_revenue) * 100.0 /
             NULLIF(previous_month_revenue, 0), 2) AS mom_growth_pct
FROM RevenueWithPrevious
ORDER BY order_year, order_month;
```

9. Power BI Data Preparation & Modelling

- Connect Power BI to the validated SQL analytical layer.
- Use Power Query for required data-type corrections and transformations.
- Create a dedicated Date table for time-intelligence analysis.
- Create relationships using the actual validated field names and compatible data types.
- Prefer SQL analytical views where appropriate rather than relying on an unprepared flat source.
- Validate Power BI totals against SQL outputs before publishing.

Core DAX measures

```
Total Revenue =
SUMX(
    Pizza_Sales,
    Pizza_Sales[quantity] * Pizza_Sales[total_price]
)
```

```
Total Orders =
DISTINCTCOUNT(Pizza_Sales[order_id])
```

```
Total Pizzas Sold =
SUM(Pizza_Sales[quantity])
```

```
Average Order Value =
DIVIDE([Total Revenue], [Total Orders], 0)
```

Time intelligence

```
Previous Month Revenue =
CALCULATE(
    [Total Revenue],
    DATEADD(DateTable[Date], -1, MONTH)
)
```

```
MoM Revenue % =
DIVIDE(
    [Total Revenue] - [Previous Month Revenue],
    [Previous Month Revenue],
    0
)
```

)

Dynamic Top-N analysis

```
TopN_Selector = GENERATESERIES(5, 20, 5)
```

```
Selected Top N =  
SELECTEDVALUE(TopN_Selector[Value], 10)
```

```
Top N Revenue =  
CALCULATE(  
    [Total Revenue],  
    TOPN(  
        [Selected Top N],  
        ALL(Pizza_Sales[pizza_name]),  
        [Total Revenue], DESC  
    )  
)
```

10. Dashboard Story & Page Design

Page	Purpose	Primary content
01 — Executive Overview	How is the business performing overall?	Revenue, Orders, Quantity, AOV; monthly revenue; MoM growth; cat
02 — Product Performance	What products and categories are driving sales?	Dynamic Top-N Revenue donut; Top 3 by category; category orders/
03 — Sales Trends & Insights	How is performance changing and what patterns are visible?	Deeper dive into trends, AOV and validated advanced analyses wh

Dashboard narrative: Page 1 establishes the business-level picture; Page 2 explains the product/category drivers behind that picture; Page 3 moves into trends, mix, and deeper analytical patterns. Visuals should not be duplicated simply to fill pages.

11. Business Insights & Evidence Standard

The project documentation intentionally separates analysis patterns from validated business findings. Final insights must be generated from the completed dashboard and reconciled with SQL results.

- Revenue trend: identify sustained increases/decreases and unusual periods.
- Product performance: identify leading and lagging pizzas by revenue and volume.
- Category contribution: compare orders, quantity, and revenue.
- Size mix: identify sizes contributing most to sales.
- Order behaviour: analyze weekday patterns and basket combinations when the required fields are valid.
- Period performance: use MoM growth for directional change and avoid unsupported YoY claims when the dataset does not provide multiple comparable years.

Current evidence status: the Notion workspace contains observations about weekday sales and a Friday-vs-Sunday order pattern, but these should be treated as provisional until the final SQL output is captured and exact values are recorded.

12. Interview Explanation

30-45 second version

I built an end-to-end Pizza Sales Analytics project using SQL Server and Power BI. I started with data-quality validation, then used T-SQL for aggregations, CTEs, window functions, ranking, reusable views and stored procedures. I connected the validated analytical layer to Power BI, created DAX measures for KPIs and period-over-period analysis, and designed a three-page dashboard covering executive performance, product performance, and sales trends. I validated the dashboard outputs

against SQL before treating any result as a business insight.

13. GitHub Publishing Structure

```
Pizza-Sales-Analytics/  
  README.md  
  SQL/  
    01_schema.sql  
    02_data_quality.sql  
    03_cleaning.sql  
    04_views.sql  
    05_analysis.sql  
  PowerBI/  
    Pizza_Sales_Dashboard.pbix  
  Documentation/  
    Project_Documentation.pdf  
    Data_Dictionary.xlsx  
  Screenshots/  
    overview.png  
    product_analysis.png  
    sales_trends.png  
  Data/  
    README.md
```

- Keep private/raw source data out of the repository when licensing or redistribution is unclear.
- Add dashboard screenshots after the final Power BI layout is complete.
- Keep SQL scripts numbered so recruiters can follow the analytical workflow.
- Use the PDF as the polished recruiter-facing documentation and Notion as the detailed working/learning record.
- Add actual GitHub and LinkedIn URLs only after they are finalized.

14. Project Status & Finalization Checklist

- ☒ SQL learning and advanced analytical patterns documented
- ☒ Data-quality approach documented
- ☒ Power BI/DAX framework documented
- ☒ Three-page dashboard story defined
- ☒ GitHub repository structure defined
- ☐ Final Power BI dashboard completed
- ☐ Final dashboard screenshots captured
- ☐ Data Dictionary finalized against the actual source
- ☐ Revenue formula validated against the meaning of total_price
- ☐ Advanced SQL outputs executed and reconciled
- ☐ Final numerical business insights recorded
- ☐ README updated with actual project evidence

Documentation source: Pizza Sales project documentation maintained in Notion under the SQL Upskilling Tracker. This PDF is a curated GitHub-facing version of that working documentation.

Prepared for GitHub portfolio use • Hanraj Hari Krishna